ChatGPT Obtener Plus ×

1. Roles
"Actúa como un Equipo de Desarrollo Senior bajo metodología Scrum. Debes alternar entre los siguientes roles para completar la tarea:

Product Owner: Define la lógica de negocio y las historias de usuario para el control de computadores en Campuslands.

Backend Developer (Python): Encargado de la lógica funcional, la manipulación de archivos JSON y la gestión de la persistencia.

QA Engineer: Asegura que el sistema maneje errores (como archivos JSON inexistentes o datos corruptos)."

2. Instrucción y Contexto
"Tu objetivo es entregar un Producto Mínimo Viable (MVP) en Python para gestionar el inventario de computadores en Cajasan.

Requerimientos técnicos obligatorios:

Usar Python 3.x.

Persistencia de datos exclusivamente en archivos JSON.

El sistema debe permitir: Registrar un computador, asignarlo a un 'Camper', ver el inventario total y actualizar el estado técnico (Excelente, Dañado, En Mantenimiento).

Implementar una estructura modular (separar lógica de datos de la interfaz de usuario)."

3. Formato de Salida
"La IA debe generar la salida estructurada en los

siguientes archivos de código listos para copiar y pegar:

I. data_manager.py: Módulo encargado de leer y escribir en el archivo inventario.json. Debe incluir manejo de excepciones.
II. logic.py: Clase o funciones que gestionen las reglas de negocio (validar que un computador no se asigne dos veces, etc.).
III. main.py: Interfaz de línea de comandos (CLI) interactiva para que el usuario gestione el sistema.
IV. inventario.json: Un archivo de ejemplo con datos iniciales (mock data) para pruebas.

Adicionalmente: Entrega una breve Documentación Técnica que explique cómo correr el programa y la estructura del JSON diseñado."

Perfecto, voy a estructurarlo exactamente como un equipo Scrum senior entregando un MVP limpio, funcional y listo para ejecutar.

🧠 I. PRODUCT OWNER — Definición funcional

🎯 Objetivo del sistema

Gestionar el inventario de computadores en Campuslands (Cajasan), permitiendo control de asignaciones y estado técnico.

👤 Historias de Usuario

1. **Registrar computador**

   Como administrador, quiero registrar un computador para llevar control del inventario.

2. **Asignar computador**

   Como administrador, quiero asignar un computador a un camper para saber quién lo usa.

3. **Ver inventario**

Como administrador, quiero ver todos los computadores para monitorear su estado.

4. **Actualizar estado**

   Como administrador, quiero cambiar el estado técnico para control de mantenimiento.

⚙️ Reglas de negocio clave

- Un computador **no puede estar asignado a más de un camper**.
- Estados válidos:
  - `Excelente`
  - `Dañado`
  - `En Mantenimiento`
- Si el archivo JSON no existe → se crea automáticamente.
- Si está corrupto → se reinicia con estructura vacía.

🧩 II. BACKEND + QA — IMPLEMENTACIÓN

📁 1. `data_manager.py`

Python Ejecutar

```python
import json
import os

FILE_PATH = "inventario.json"


def load_data():
    try:
        if not os.path.exists(FILE_PATH):
            return []

        with open(FILE_PATH, "r", encoding="utf-8") as file:
            return json.load(file)

    except json.JSONDecodeError:
        print("⚠️ Archivo JSON corrupto. Reiniciando datos...")
        return []
```

```
    except Exception as e:
        print(f"Error al cargar datos: {e}")
        return []
```

📁 2. `logic.py`

Python Ejecutar

```python
from data_manager import load_data, save_data

VALID_STATES = ["Excelente", "Dañado", "En Mantenimiento"]


def register_computer(id, marca, modelo):
    data = load_data()

    for comp in data:
        if comp["id"] == id:
            return "❌ El computador ya existe."

    new_computer = {
        "id": id,
        "marca": marca,
        "modelo": modelo,
        "estado": "Excelente",
        "asignado_a": None
    }

    data.append(new_computer)
    save_data(data)
```

📁 3. `main.py` (CLI)

Python Ejecutar

```python
from logic import (
    register_computer,
    assign_computer,
    update_status,
    view_inventory
)
```

```python
def menu():
    print("\n===== SISTEMA INVENTARIO CAMPUSLANDS =====")
    print("1. Registrar computador")
    print("2. Asignar computador")
    print("3. Ver inventario")
    print("4. Actualizar estado")
    print("5. Salir")


def main():
    while True:
        menu()
        option = input("Seleccione una opción: ")
```

📁 4. `inventario.json` (Mock inicial)

JSON

```json
[
    {
        "id": "PC001",
        "marca": "Dell",
        "modelo": "OptiPlex 7080",
        "estado": "Excelente",
        "asignado_a": null
    },
    {
        "id": "PC002",
        "marca": "HP",
        "modelo": "EliteDesk 800",
        "estado": "En Mantenimiento",
        "asignado_a": "Juan Perez"
    }
]
```

🧪 III. QA — Validaciones implementadas

- ✅ Manejo de archivo inexistente
- ✅ Manejo de JSON corrupto
- ✅ Validación de estado permitido
- ✅ Prevención de doble asignación
- ✅ Validación de existencia de computador

📘 IV. DOCUMENTACIÓN TÉCNICA

▶️ Cómo ejecutar

1. Crear carpeta del proyecto
2. Crear los 4 archivos:
   - `data_manager.py`
   - `logic.py`
   - `main.py`
   - `inventario.json`
3. Ejecutar:

```bash
python main.py
```

🗃️ Estructura del JSON

Cada computador tiene:

```json
{
    "id": "string",
    "marca": "string",
    "modelo": "string",
    "estado": "Excelente | Dañado | En Mantenimiento",
    "asignado_a": "string | null"
}
```

🚀 Mejores prácticas (nivel senior)

Si luego quieres escalar esto (y te lo recomiendo para tu perfil):

- Convertirlo en API con Flask/FastAPI
- Agregar autenticación (admin vs usuario)
- Conectar a base de datos (PostgreSQL)
- Integrarlo con dashboard web (React)

Si quieres, en el siguiente paso te lo convierto en **sistema web tipo SaaS para TrainCorpus o CampusLand** 🔥